



第17课时：函数的定义与调用

封装 · 复用 · 模块化

函数的作用、定义语法、参数传递与 void 函数



上节课回顾



上节课我们学习了字符串高级操作：

- **字符串比较**：用 `==`、`<`、`<=` 等关系运算符按字典序比较
- **子串查找**：`s.find(sub)`、`s.rfind(sub)`，未找到返回 `string::npos`
- **子串提取**：`s.substr(pos, len)` 提取子串
- **ASCII 码**：`'A'` = 65、`'a'` = 97、`'0'` = 48，大小写相差 32
- **字符串遍历**：下标循环 `s[i]` 或范围 `for (char c : s)`

本节课我们将进入**函数与结构化编程**——这是 C++ 编程的核心思想之一。

为什么需要函数?



问题： 如果要求你写一个程序，输出"你好，世界！"三次，怎么办？

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    cout << "你好，世界！" << endl;
    cout << "你好，世界！" << endl;
    cout << "你好，世界！" << endl;
    return 0;
}
```

看起来还行。但如果是 **100 次** 呢？ **1000 次** 呢？

💡 **重复的代码，既冗长又难维护**——如果想改成"Hello, World!", 要改 1000 处。

函数的引入



使用**函数**封装重复代码：

```
#include <bits/stdc++.h>
using namespace std;

// 定义一个函数：打印"你好，世界！"
void sayHello() {
    cout << "你好，世界！" << endl;
}

int main() {
    // 调用 1000 次只需要改这一行
    for (int i = 0; i < 1000; i++) {
        sayHello();
    }
    return 0;
}
```

✓ 函数把"一段代码"起个名字，**需要时调用即可**。

函数的作用



三大核心作用：

1. **代码复用**：写一次，用多次（避免重复代码）
2. **模块化**：把大问题拆成小问题，分别解决
3. **可读性**：用有意义的函数名表达意图

类比：

- 就像数学里的 $f(x) = 2x + 1$ —— 给一个公式起个名字 f ，输入 x 得到结果
- 就像工厂的"模具"——做好模具后，可以批量生产相同的产品

函数定义的基本语法



```
返回值类型  函数名(参数列表) {  
    // 函数体  
    return 返回值;    // 如果有返回值  
}
```

四个部分缺一不可：

部分	说明	示例
返回值类型	函数返回的数据类型	<code>int</code> 、 <code>void</code> 、 <code>double</code>
函数名	自己起的标识符	<code>add</code> 、 <code>printMax</code>
参数列表	输入数据，可为空	<code>(int a, int b)</code> 或 <code>()</code>
函数体	具体逻辑	<code>{ ... }</code>



第一个函数：求两数之和



```
#include <bits/stdc++.h>
using namespace std;

// 定义：计算 a + b
int add(int a, int b) {
    int sum = a + b;
    return sum;
}

int main() {
    int x = 3, y = 5;
    int result = add(x, y);    // 调用
    cout << result << endl;  // 输出 8
    return 0;
}
```

解读：

- `int add(int a, int b)`：返回值 `int`，函数名 `add`，两个 `int` 参数
- `return sum`：把 `sum` 的值返回给调用者
- `add(x, y)`：实参 `x`、`y` 传递给形参 `a`、`b`

函数的执行过程



调用 `add(3, 5)` 时发生了什么？

```
1. 实参准备    3, 5  —> 形参赋值    a = 3, b = 5
                                   ↓
2. 执行函数体    sum = 3 + 5 = 8
                                   ↓
3. 返回结果    return 8  —> result = 8
                                   ↓
4. 继续执行    cout << result << endl;
```

💡 调用函数时，程序会"跳到"函数处执行，执行完 `return` 后"跳回"调用处继续。

无返回值——void 函数



有时函数不需要返回值，用 `void` 关键字：

```
#include <bits/stdc++.h>
using namespace std;

void greet(string name) {           // 无返回值
    cout << "你好, " << name << "!" << endl;
    // 没有 return 语句 (或写 return;)
}

int main() {
    greet("张三");    // 输出: 你好, 张三!
    greet("李四");    // 输出: 你好, 李四!
    return 0;
}
```

⚠ **注意：** `void` 函数中**不能** `return` 值，只能 `return;`（或不写）。

多个参数的函数



函数可以有多个参数，用逗号分隔：

```
#include <bits/stdc++.h>
using namespace std;

// 求三个数的最大值
int max3(int a, int b, int c) {
    int m = a;
    if (b > m) m = b;
    if (c > m) m = c;
    return m;
}

int main() {
    cout << max3(3, 7, 5) << endl;    // 7
    cout << max3(10, 2, 8) << endl;   // 10
    return 0;
}
```

调用规则：实参个数、类型、顺序必须与形参**一一对应**。

参数传递——值传递（默认）



C++ 默认采用**值传递**：形参是实参的"副本"，修改形参不影响实参。

```
#include <bits/stdc++.h>
using namespace std;

void tryModify(int x) {
    x = 100;    // 只改副本
    cout << "函数内 x = " << x << endl;
}

int main() {
    int a = 10;
    tryModify(a);
    cout << "函数外 a = " << a << endl;
    return 0;
}

// 输出：
// 函数内 x = 100
// 函数外 a = 10    ← 实参没变!
```

💡 **重要**：函数内的修改**不会**影响调用者的变量。

多个返回值? ——通过引用参数 (了解)



若希望函数"返回"多个值, 可用**引用参数** `&` (进阶内容, 简单了解):

```
#include <bits/stdc++.h>
using namespace std;

// 求 a/b 的商和余数
void divide(int a, int b, int &quot, int &rem) {
    quot = a / b;
    rem = a % b;
}

int main() {
    int q, r;
    divide(17, 5, q, r);
    cout << q << " " << r << endl;    // 3 2
    return 0;
}
```

⚠ `"` 表示引用——传入的是变量本身, 函数内可修改。本节课不深入, 下节课讲解。

函数的声明



如果函数定义在调用之后，需要**先声明**：

```
#include <bits/stdc++.h>
using namespace std;

// 函数声明（原型）
int add(int a, int b);    // 以分号结尾

int main() {
    cout << add(3, 5) << endl;
    return 0;
}

// 函数定义（完整实现）
int add(int a, int b) {
    return a + b;
}
```

💡 **声明**告诉编译器“有这么个函数”，**定义**才是真正的实现。

函数的位置约定



通常把函数放在 `main` 之前，或者用**声明 + 定义分离**：

```
// 方案 A: 所有函数在 main 之前
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int main() { /* 使用 add, sub */ }
```



```
// 方案 B: 声明在 main 之前，定义在 main 之后
int add(int a, int b);
int sub(int a, int b);
int main() { /* 使用 add, sub */ }
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
```

两种方式都可以，按团队习惯选择。

例题 1：求最大值



题目描述

输入两个整数，输出较大的那个。

输入输出示例：

- 输入：5 8
- 输出：较大值：8

解题思路

1. 定义函数 `getMax(int a, int b)`，返回 `a`、`b` 中的较大值
2. 用 **三元运算符** `(a > b) ? a : b` 实现比较，一行搞定
3. `main` 中读入两个整数，调用 `getMax` 输出结果

例题 1：求最大值 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int getMax(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int x, y;
    cin >> x >> y;
    cout << "较大值: " << getMax(x, y) << endl;
    return 0;
}
// 输入: 5 8
// 输出: 较大值: 8
```

💡 用三元运算符 `? :` 一行解决，简洁优雅。

例题 2：判断素数



题目描述

输入一个正整数 `n`，判断它是否为素数（只能被 1 和自身整除）。

输入输出示例：

- 输入：7 → 输出：7 是素数
- 输入：9 → 输出：9 不是素数

解题思路

1. 定义函数 `bool isPrime(int n)`，返回 `true` / `false`
2. 特判：`n < 2` 直接返回 `false`
3. 从 `i = 2` 试除到 `i * i <= n`（只需到 $\sqrt{n}$ 即可）
4. 若 `n % i == 0` 则非素数，返回 `false`；否则返回 `true`
5. `main` 中读入 `n`，调用 `isPrime` 输出结论

例题 2：判断素数 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    int n;
    cin >> n;
    if (isPrime(n)) {
        cout << n << " 是素数" << endl;
    } else {
        cout << n << " 不是素数" << endl;
    }
    return 0;
}
```

例题 3：打印矩形



题目描述

用函数打印一个由 `*` 组成的 `m` 行 `n` 列矩形。

输入输出示例：

- 调用 `printRect(3, 5)`，输出：

```
* * * * *  
* * * * *  
* * * * *
```

解题思路

1. 定义 **void 函数** `printRect(int m, int n)`，无需返回值
2. 外层循环 `i` 从 0 到 `m-1`，控制行数
3. 内层循环 `j` 从 0 到 `n-1`，每行打印 `n` 个 `*`
4. 每行结束后输出 `endl` 换行
5. `main` 中直接调用即可

例题 3：打印矩形 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

void printRect(int m, int n) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cout << "*" << " ";
        }
        cout << endl;
    }
}

int main() {
    printRect(3, 5);
    return 0;
}

// 输出:
// * * * * *
// * * * * *
// * * * * *
```

例题 4: 求 $1+2+\dots+n$



题目描述

输入正整数 n ，求 $1 + 2 + 3 + \dots + n$ 的和。

输入输出示例：

- 输入：100 → 输出：5050
- 输入：10 → 输出：55

解题思路

1. 定义函数 `int sumToN(int n)`，返回 1 到 n 的累加和
2. 初始化变量 `s = 0`
3. 循环 `i` 从 1 到 n ，每次 `s += i`
4. 返回 `s`
5. `main` 中读入 n ，调用 `sumToN` 输出结果

⚠ 也可以用公式 $n*(n+1)/2$ ，但这里重点练习函数封装循环逻辑。

例题 4：求 $1+2+\dots+n$ —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int sumToN(int n) {
    int s = 0;
    for (int i = 1; i <= n; i++) {
        s += i;
    }
    return s;
}

int main() {
    int n;
    cin >> n;
    cout << sumToN(n) << endl;
    return 0;
}

// 输入: 100
// 输出: 5050
```

常见错误与调试



错误类型	示例/说明	解决方法
忘记返回值	非 <code>void</code> 函数漏写 <code>return</code>	编译器警告，确保每条路径都有
实参形参类型不符	<code>add(3.5, 5)</code> 传给 <code>int</code> 形参	类型必须匹配，或强制转换
实参形参个数不符	<code>add(3)</code> 只传 1 个参数	实参个数必须等于形参
函数未声明	在 <code>main</code> 后定义且未声明	加函数声明，或把定义提前
<code>void</code> 函数 <code>return</code> 值	<code>return 5;</code> 在 <code>void</code> 函数中	只写 <code>return;</code> 或不写
形参实参混淆	以为形参修改能影响实参	值传递只改副本；用引用才改原值



知识小结



✓ **函数定义**: 返回值类型 函数名(参数列表) { 函数体; return 返回值; }

✓ **函数调用**: 用 函数名(实参) 调用, 实参传给形参

✓ **返回值**: 通过 return 返回; void 表示无返回值

✓ **参数传递**: 默认**值传递**, 形参是实参的副本, 不影响原值

✓ **函数声明**: 用分号结尾的"原型", 让编译器先认识再调用

✓ **函数好处**: 代码复用、模块化、提高可读性

下节课预告



下一课时：函数进阶——递归入门（第18课时）

- 递归的概念：函数调用自身
- 递归的两要素：**边界条件** + **递归公式**
- 经典递归示例：**阶乘**、**斐波那契数列**
- 递归的优缺点与常见错误

"把大问题拆成同类型的更小问题，是递归的灵魂。"

谢谢大家！



函数是 C++ 编程的基石，掌握它就能写出结构清晰、易于维护的程序！

Q & A